CHAPTER – 4

ANALYZING FUNCTIONAL DEPENDENCIES, REDUNDANCY AND DATA ANOMALIES

4.1 INTRODUCTION :

Functional Dependency is an important concept in database design that defines the relationship between attributes in a table. It helps reduce data redundancy and maintain data consistency. In the SHOPSY Online E-Commerce Database Management System, functional dependencies are analyzed for entities such as Customer, Seller, Category, Product, Cart, Order, Order_Item, Payment, Inventory, Delivery, and Review. This analysis helps identify redundancy, prevent data anomalies, and ensure an efficient and well-structured database.

4.2.1 CUSTOMER :

Functional Dependency

Customer_ID → Customer_Name, Email, Phone_Number, Address, Password

Redundancy Analysis

- Customer details are stored only once using Customer_ID, reducing duplicate customer information.

Data Anomalies

- **Insertion Anomaly:** Customer details cannot be stored without creating a Customer_ID.
- **Update Anomaly:** Updating customer email or phone number in multiple records may cause inconsistency.
- **Deletion Anomaly:** Deleting a customer record removes all customer information.

4.2.2 SELLER :

Functional Dependency

Seller_ID → Seller_Name, Email, Phone_Number, Address

Redundancy Analysis

- Seller information is stored only once using Seller_ID, avoiding duplicate seller records.

Data Anomalies

- **Insertion Anomaly:** Seller information cannot be stored without a Seller_ID.
- **Update Anomaly:** Updating seller details in multiple records may cause inconsistency.
- **Deletion Anomaly:** Deleting a seller record removes all seller information.

4.2.3 CATEGORY :

Functional Dependency

Category_ID → Category_Name, Description

Redundancy Analysis

- Category information is stored only once and linked with products.

Data Anomalies

- **Insertion Anomaly:** A category cannot be created without a Category_ID.
- **Update Anomaly:** Updating the category name in multiple records may cause inconsistency.
- **Deletion Anomaly:** Deleting a category may affect related products.

4.2.4 PRODUCT :

Functional Dependency

Product_ID → Product_Name, Category_ID, Price, Stock, Color, Size

Redundancy Analysis

- Product details are stored only once using Product_ID, avoiding duplicate product information.

Data Anomalies

- **Insertion Anomaly:** Product details cannot be added without a Product_ID.

- **Update Anomaly:** Updating product price or stock in multiple records may cause inconsistency.
- **Deletion Anomaly:** Deleting a product removes all product information.

4.2.5 CART :

Functional Dependency

Cart_ID → Customer_ID, Product_ID, Quantity, Total_Amount

Redundancy Analysis

- Cart details are maintained separately to avoid duplicate cart entries.

Data Anomalies

- **Insertion Anomaly:** A cart cannot be created without customer and product details.
- **Update Anomaly:** Updating cart quantity in multiple places may cause inconsistency.
- **Deletion Anomaly:** Deleting a cart removes all selected products.

4.2.6 ORDER :

Functional Dependency

Order_ID → Customer_ID, Order_Date, Grand_Total, Order_Status

Redundancy Analysis

- Order information is stored only once and linked with customers.

Data Anomalies

- **Insertion Anomaly:** An order cannot be created without customer information.
- **Update Anomaly:** Updating order status in multiple records may cause inconsistency.
- **Deletion Anomaly:** Deleting an order removes the order history.

4.2.7 ORDER_ITEM :

Functional Dependency

Order_Item_ID → Order_ID, Product_ID, Quantity, Subtotal

Redundancy Analysis

- Order item details are stored separately to avoid duplicate order records.

Data Anomalies

- **Insertion Anomaly:** Order items cannot exist without an Order_ID.
- **Update Anomaly:** Updating quantity or subtotal may cause inconsistency.
- **Deletion Anomaly:** Deleting an order item removes product details from the order.

4.2.8 PAYMENT :

Functional Dependency

Payment_ID → Order_ID, Payment_Method, Payment_Status, Payment_Date

Redundancy Analysis

- Payment details are stored separately to avoid duplicate payment information.

Data Anomalies

- **Insertion Anomaly:** Payment cannot be recorded without an order.
- **Update Anomaly:** Updating payment status in multiple records may cause inconsistency.
- **Deletion Anomaly:** Deleting a payment record removes transaction history.

4.2.9 INVENTORY :

Functional Dependency

Inventory_ID → Product_ID, Stock_Quantity, Last_Updated

Redundancy Analysis

- Inventory details are maintained separately to avoid duplicate stock records.

Data Anomalies

- **Insertion Anomaly:** Inventory cannot exist without a Product_ID.
- **Update Anomaly:** Updating stock quantity in multiple records may cause inconsistency.
- **Deletion Anomaly:** Deleting inventory removes stock information.

4.2.10 DELIVERY :

Functional Dependency

Delivery_ID → Order_ID, Delivery_Address, Delivery_Status, Delivery_Date

Redundancy Analysis

- Delivery information is stored separately, reducing duplicate delivery records.

Data Anomalies

- Insertion Anomaly: Delivery details cannot exist without an order.
- Update Anomaly: Updating delivery status in multiple records may cause inconsistency.
- Deletion Anomaly: Deleting a delivery record removes delivery history.

4.2.11 REVIEW :

Functional Dependency

Review_ID → Customer_ID, Product_ID, Rating, Review_Text, Review_Date

Redundancy Analysis

- Review details are stored only once using Review_ID, avoiding duplicate reviews.

Data Anomalies

- **Insertion Anomaly:** A review cannot be added without customer and product details.
- **Update Anomaly:** Updating review information in multiple records may cause inconsistency.
- **Deletion Anomaly:** Deleting a review removes customer feedback.

CONCLUSION :

The functional dependency analysis of the SHOPSY Online E-Commerce Database Management System shows that each entity has a clear relationship between its primary key and non-key attributes. Proper normalization reduces redundancy, prevents insertion, update, and deletion anomalies, and improves data integrity, consistency, and efficiency. This database design provides a reliable and well-structured system for managing customers, sellers, categories, products, carts, orders, order items, payments, inventory, deliveries, and reviews.